

BigBasket E-Commerce Automation Testing

Selenium Pytest and BDD Behave Automation Suite

Student Name	Sudabattula Sahithi
Superset ID	5500933
Organization	Wipro
Project Type	Capstone Project
Application Under Test	BigBasket web application
Project Name	Automation Testing using Selenium Pytest Behave BDD Framework
Domain	Search and Order
Module Title	BigBasket

This report documents two automation implementations for the same e-commerce user journey: a Selenium pytest project and a BDD Behave project. The report explains the objective, strategy, tools, environment, execution flow, and evidence captured from screenshots, logs, and Allure reports.

Section	Contents
1	Synopsis
2	Project Description
3	Test Strategy
4	Test Environment
6	Test Execution
6	Test Outputs
7	Test Reports

1. Synopsis

This project is developed to automate important workflows of the BigBasket e-commerce website using Selenium with Python. BigBasket was selected as the application under test because it provides real-time online shopping functionalities such as login, product search, adding items to the basket, basket validation, and checkout navigation. These activities are repeatedly executed during testing and are ideal for automation since they help reduce manual effort, improve testing speed, and ensure application reliability during regression testing.

The project mainly focuses on validating the core shopping functionality through automated test cases. Different types of scenarios such as positive testing, negative testing, and end-to-end workflow testing are included to ensure the application behaves correctly under different conditions.

2. Project Description

The project consists of two separate automation implementations created for the same BigBasket module. The first implementation, named **bigB**, is developed using Selenium WebDriver with pytest framework. In this approach, Python test functions are used along with the Page Object Model (POM) design pattern to keep browser actions reusable, maintainable, and organized.

The second implementation, named **BDD_BigBasket**, is developed using the Behavior Driven Development (BDD) approach with Behave framework. In this version, test scenarios are written in simple Gherkin language, making the test flow easy to understand for both technical and non-technical users. These scenarios are connected to Python step definition files that execute the automation steps.

Both automation projects validate multiple workflows of the application. Positive test cases verify whether valid products can be searched and successfully added to the basket. Negative test cases check how the system behaves for invalid searches and empty basket checkout attempts. End-to-end test cases validate the complete user journey, including login, product search, add product to basket, navigates to the basket and proceeds to the checkout page.

3. Test Strategy

Selenium Pytest Project

The Selenium automation project uses Selenium WebDriver with Python and PyTest framework for browser automation testing. The framework is designed using the Page Object Model (POM), where separate page classes are maintained for login, search, basket, and checkout functionalities. Test data is handled using CSV files, which helps in implementing data-driven testing and keeps test inputs separate from the test scripts. Reporting is generated using Allure Report and pytest-html, while custom logging and screenshot capture mechanisms are used to store execution evidence for passed and failed test cases.

FEATURES SUPPORTED

- End-to-End Testing
- Positive Testing
- Negative Testing
- Data-Driven Testing
- Screenshot Capture
- Logging and Reporting

TOOLS USED

- Selenium WebDriver
- Python
- PyTest
- CSV Reader
- Allure Report
- pytest-html

BDD Behave Project

The BDD automation project is implemented using Behave framework with Gherkin feature files. Test scenarios are written in simple business-readable language, making them easy to understand for both technical and non-technical users. The step definition files are connected to reusable Page Object Model methods similar to the Selenium PyTest project. The framework supports automated reporting using Allure Behave, and environment hooks are implemented to automatically capture screenshots and attach logs during scenario execution.

FEATURES SUPPORTED

- Behavior Driven Development (BDD)
- End-to-End Testing
- Positive Testing
- Negative Testing
- Data-Driven Testing
- Screenshot Capture
- Logging and Reporting

TOOLS USED

- Selenium WebDriver
- Python
- Behave BDD
- Gherkin
- Allure Behave
- Page Object Model (POM)

4. Test Environment

Category	Details
Operating System	Windows 11
IDE	PyCharm 2026.1.2
Language	Python 3.13.x
Browser	Microsoft Edge
Automation Library	Selenium WebDriver
Selenium Project Framework	pytest, pytest-html, allure-pytest
BDD Project Framework	behave, allure-behave, Gherkin feature files
Design Pattern	Page Object Model with reusable page classes
Test Data	CSV files: login_data.csv, positive_data.csv, negative_data.csv, search_data.csv
Reports and Evidence	Allure results, HTML report, screenshots folder, logs/automation.log

FRAMEWORK ARCHITECTURE FOR SELENIUM PYTEST

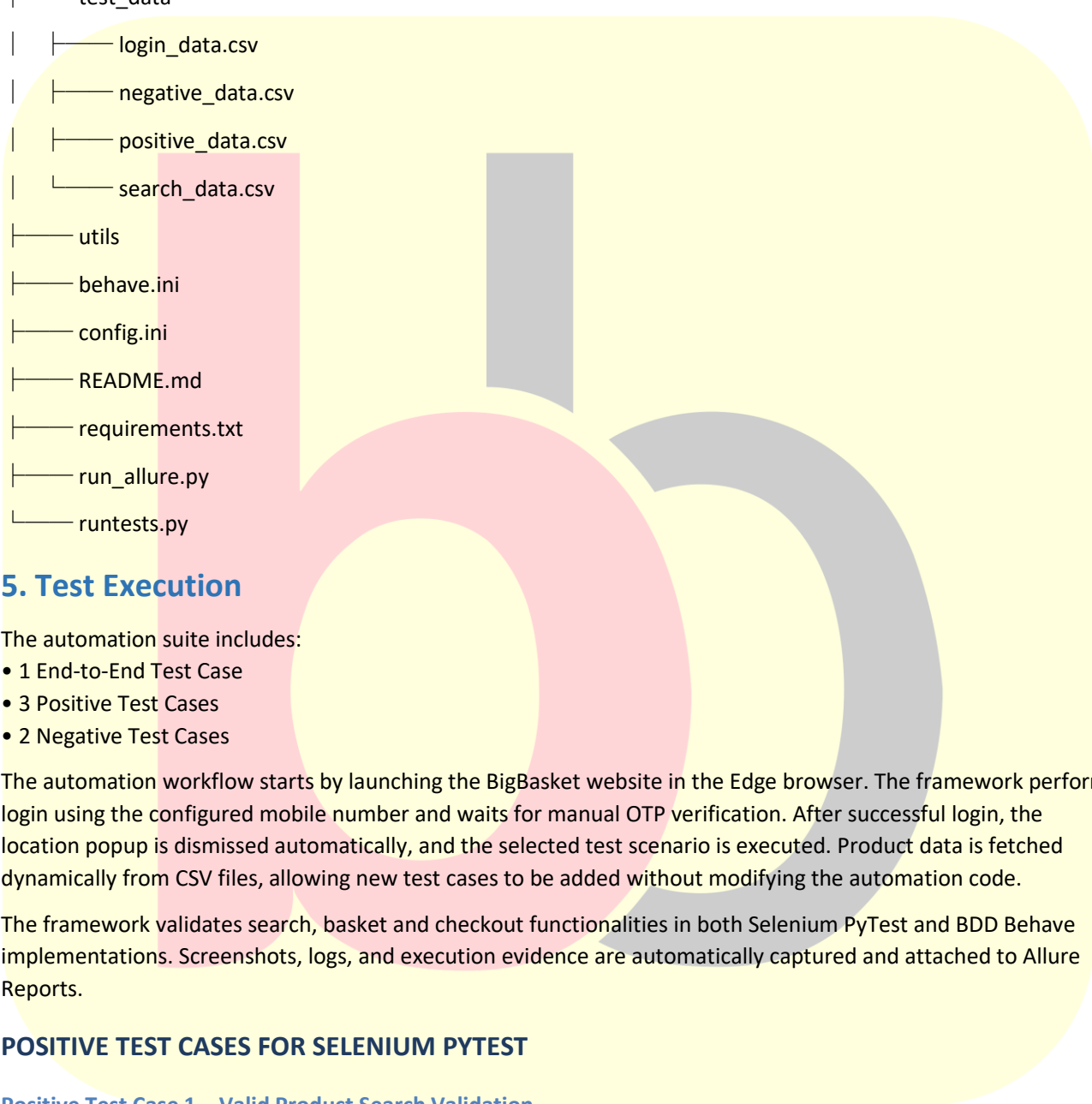
bigB [BigBasket]

```
|— logs
|  |— automation.log
|— pages
|  |— __init__.py
|  |— base_page.py
|  |— login_page.py
|  |— search_page.py
|— reports
|— screenshots
```

```
├── test_data
│   ├── login_data.csv
│   ├── negative_data.csv
│   ├── positive_data.csv
│   └── search_data.csv
├── tests
│   ├── __init__.py
│   ├── test_negative.py
│   ├── test_positive.py
│   └── test_search_and_checkout.py
├── utils
├── conftest.py
├── main.py
├── pytest.ini
└── requirements.txt
```

FRAMEWORK ARCHITECTURE FOR BDD WITH BEHAVE

```
BDD_BigBasket
├── features
│   ├── 01_negative.feature
│   ├── 02_positive.feature
│   ├── 03_end_to_end.feature
│   └── steps
│       ├── cart_steps.py
│       ├── login_steps.py
│       ├── product_steps.py
│       └── purchase_steps.py
├── environment.py
├── logs
│   └── automation.log
├── pages
│   ├── __init__.py
│   └── base_page.py
```



```
| |—— login_page.py
| |—— search_page.py
|—— reports
|—— screenshots
|—— test_data
| |—— login_data.csv
| |—— negative_data.csv
| |—— positive_data.csv
| |—— search_data.csv
|—— utils
|—— behave.ini
|—— config.ini
|—— README.md
|—— requirements.txt
|—— run_allure.py
|—— runtests.py
```

5. Test Execution

The automation suite includes:

- 1 End-to-End Test Case
- 3 Positive Test Cases
- 2 Negative Test Cases

The automation workflow starts by launching the BigBasket website in the Edge browser. The framework performs login using the configured mobile number and waits for manual OTP verification. After successful login, the location popup is dismissed automatically, and the selected test scenario is executed. Product data is fetched dynamically from CSV files, allowing new test cases to be added without modifying the automation code.

The framework validates search, basket and checkout functionalities in both Selenium PyTest and BDD Behave implementations. Screenshots, logs, and execution evidence are automatically captured and attached to Allure Reports.

POSITIVE TEST CASES FOR SELENIUM PYTEST

Positive Test Case 1 – Valid Product Search Validation

- Validates successful product search functionality after login
- Searches products dynamically using CSV test data
- Verifies successful search execution for products like “Cricket bat”
- Ensures search operation is completed without errors

Positive Test Case 2 – Add Product to Basket Validation

- Validates add-to-basket functionality
- Adds searched product into the basket successfully
- Verifies successful product addition workflow
- Ensures selected product is added to the shopping basket

Positive Test Case 3 – Basket Open Validation

- Validates basket opening functionality
- Opens basket after adding selected product
- Verifies successful basket navigation
- Ensures added product is visible inside basket section

Positive Test Case 4 – Checkout Navigation Validation

- Validates checkout workflow after basket operation
- Opens checkout process from basket page
- Verifies successful checkout navigation flow
- Ensures positive test execution completes successfully for selected product

NEGATIVE TEST CASES FOR SELENIUM PYTEST

Negative Test Case 1 – Invalid Product Search Validation

- Searches using invalid or unavailable product names
- Validates application response handling
- Verifies no matching product results are displayed
- Ensures invalid search inputs are handled properly

Negative Test Case 2 – Empty Basket Checkout Validation

- Attempts checkout with an empty basket
- Validates basket validation functionality
- Verifies checkout restriction behavior
- Ensures checkout is not allowed without products in basket

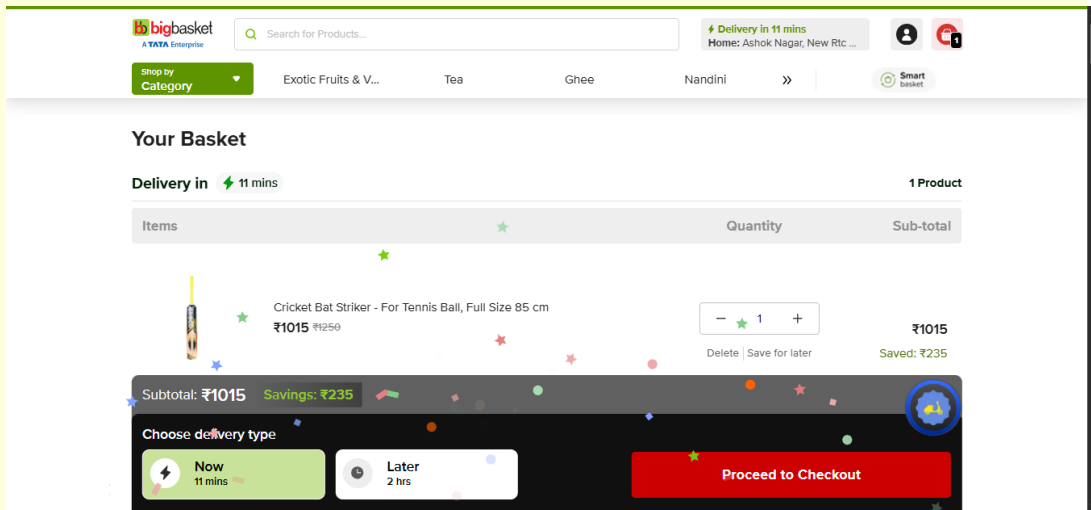
END-TO-END TEST CASE

End-to-End Test Case – Complete Shopping Workflow Validation

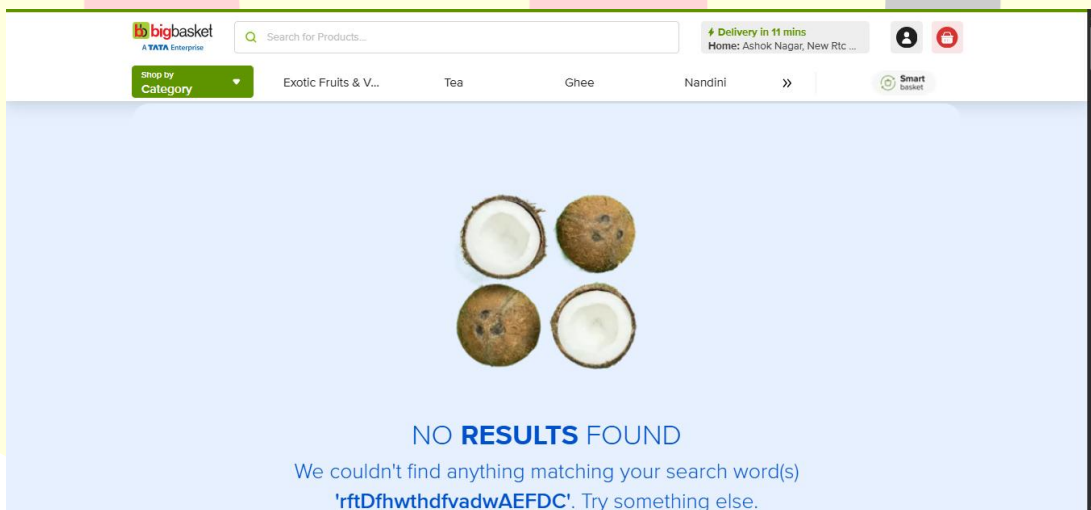
- Validates complete shopping workflow from login to checkout
- Performs login using mobile number and OTP verification
- Searches valid product dynamically from CSV data
- Opens basket and proceeds to checkout page
- Verifies successful checkout navigation workflow

6. Test Outputs

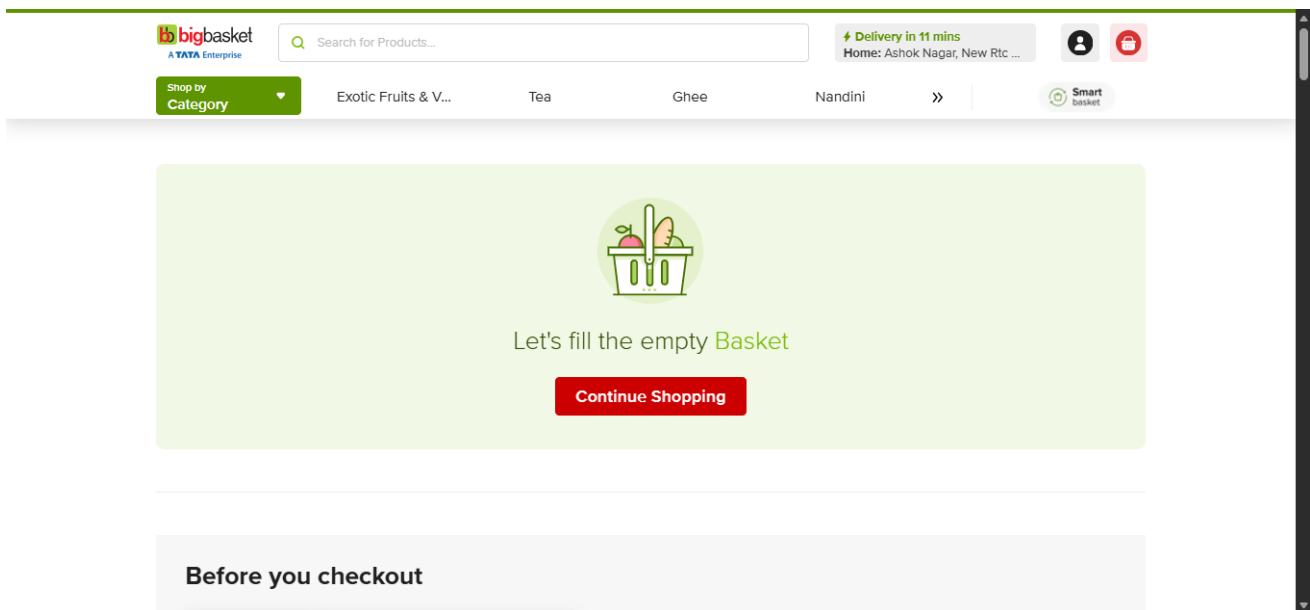
Screenshots - Positive, Negative, and End-to-End Evidence



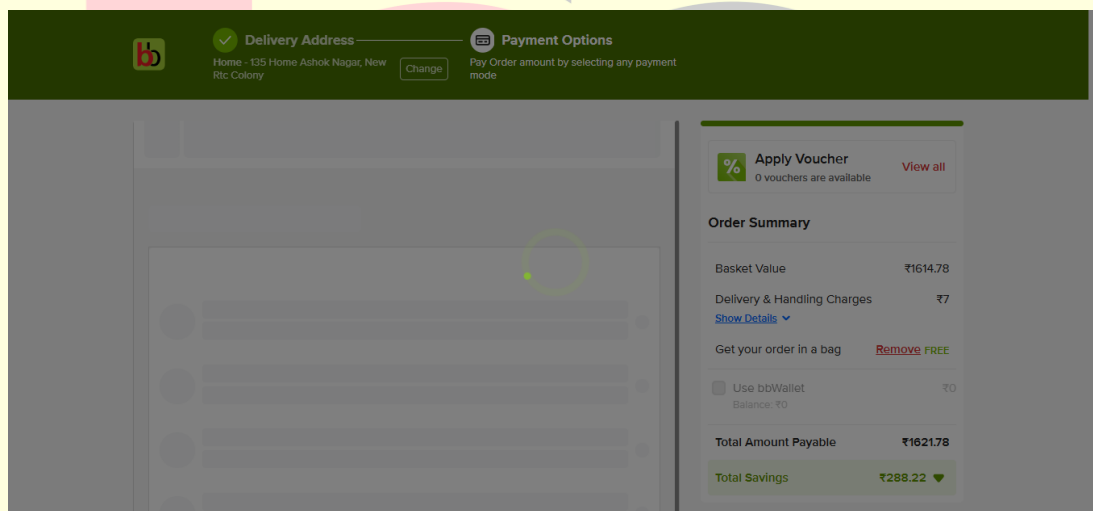
Selenium pytest and BDD behave positive flow: product added to basket



Selenium pytest and BDD behave negative flow: invalid search screenshot



Selenium pytest and BDD behave negative flow: empty basket screenshot



Selenium pytest and BDD behave end-to-end flow: checkout test passed screenshot

Log File Sample

Selenium pytest log sample:

```

2026-05-22 12:33:20,957 - INFO - Verify & Continue clicked
2026-05-22 12:33:20,958 - INFO - Checking for location popup after login
2026-05-22 12:33:23,363 - INFO - Location popup dismissed – clicked 'Got it'
2026-05-22 12:34:22,124 - INFO - === Test PASSED | mobile: 9030652433 | product: Amul Butter ===
2026-05-22 12:34:22,125 - INFO - Test PASSED: test_search_and_checkout[9030652433-Amul Butter] - capturing
screenshot
2026-05-22 12:34:22,224 - INFO - Screenshot saved: screenshots\PASSED_test_search_and_checkout_9030652433-
Amul_Butter.png
2026-05-22 12:34:22,239 - INFO - Quitting WebDriver
2026-05-22 12:34:29,522 - INFO - === WebDriver session closed ===

```

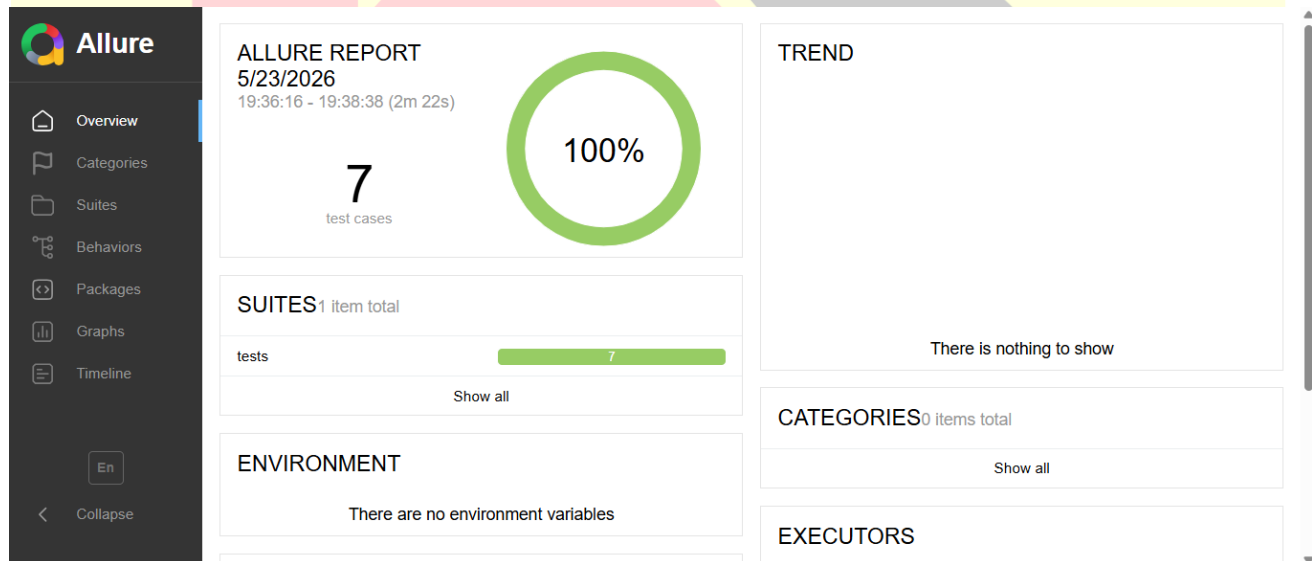
BDD Behave log sample:

```
2026-05-22 13:10:32,394 - INFO - Location popup dismissed - clicked 'Got it'
2026-05-22 13:10:32,424 - INFO - Reading CSV:
C:\Users\sahii\OneDrive\Desktop\CapPro\BDD_BigBasket\test_data\search_data.csv
2026-05-22 13:10:32,424 - INFO - CSV loaded: 2 rows from search_data.csv
2026-05-22 13:10:51,187 - INFO - SCENARIO STATUS: Status.passed
2026-05-22 13:10:51,319 - INFO - Screenshot saved:
screenshots\Login_search_csv_products_and_navigate_to_checkout_20260522_131051.png
2026-05-22 13:10:58,602 - INFO - Browser closed
2026-05-22 13:10:58,603 - INFO - =====
2026-05-22 13:11:00,163 - INFO - Allure HTML report generated: reports\allure-report
2026-05-22 13:11:00,164 - INFO - Report shortcut generated: reports\report.html
2026-05-22 13:11:00,172 - INFO - Allure report opened automatically
```

Allure Report Evidence

Allure reports are generated under each project's reports/allure-results folder. The BDD project also generates reports/allure-report and opens the report automatically at the end of the behave command. Screenshots and scenario-specific logs are attached to the Allure report, making it easier to verify what happened during each test run.

Allure Reports for selenium with pytest



Overview

Categories

Suites

Behaviors

Packages

Graphs

Timeline

En

Collapse

Status: 0 0 7 0 0

Marks: [icon] [icon] [icon] [icon] [icon]

tests 7

test_negative 2

#2 test_negative_empty_basket_checkout 14s 267ms

#1 test_negative_invalid_search[rtDfwilfWAEFDG]s 344ms

test_positive 4

TestPositiveBigBasket 4

#2 test_add_to_basket[Cricket {product: 'Cricket... bat}] 1s 827ms

#4 test_checkout_button_visible[Cricket {product: 'Cricket... bat}] 20ms

#3 test_open_basket[Cricket {product: 'Cricket... bat}] 4s 741ms

#1 test_search_product[Cricket {product: 'Cricket... bat}] 20s 281ms

test_search_and_checkout 1

#1 test_search_and_checkout[9D306524334652433... Amul Butter] 48s 786ms

No item selected

Allure Reports for BDD with behave

Overview

Categories

Suites

Behaviors

Packages

Graphs

Timeline

En

Collapse

ALLURE REPORT

5/23/2026

18:20:37 - 18:23:11 (2m 33s)

7

test cases

100%

SUITES 7 items total

Show all

ENVIRONMENT

Project	BDD_BigBasket
Application	BigBasket
Browser	edge
BaseURL	https://www.bigbasket.com/

TREND

There is nothing to show

CATEGORIES 0 items total

Show all

EXECUTORS

There is no information about tests executors

10

The screenshot shows the Allure web interface. On the left is a dark sidebar with navigation links: Overview, Categories, Suites (selected), Behaviors, Packages, Graphs, and Timeline. The main area is titled 'Suites' and displays a table of test results. The table has columns for 'order', 'name', 'duration', and 'status'. All 7 tests are marked with a green checkmark, indicating they passed. The tests are:

order	name	duration	status
#4	Add positive product to basket	8s 645ms	Passed
#6	Checkout option should be visible for basket with product	13s 097ms	Passed
#2	Empty basket checkout should be blocked	7s 837ms	Passed
#1	Invalid product search should not show add button	38s 226ms	Passed
#7	Login search single csv product and navigate to checkout	30s 019ms	Passed
#5	Open basket after adding positive product	12s 088ms	Passed
#3	Search positive product from csv	19s 797ms	Passed

At the top right of the main area, it says 'No item selected'. The bottom of the sidebar has a 'Collapse' button.

7. Test Reports

- ✓ Total Tests Executed: 7
- ✓ Tests Passed: 7
- ✓ Tests Failed: 0
- ✓ Automation Accuracy: 100%

Conclusion

This project successfully automated the major workflows of the BigBasket e-commerce website using Selenium PyTest and BDD Behave frameworks. The automation suite validated functionalities such as login, product search, add-to-basket, basket validation, and checkout navigation with successful execution results. The use of Page Object Model (POM), CSV-based test data, logging, screenshot capture, and Allure reporting helped create a structured, reusable, and maintainable automation framework. Overall, the project reduced manual testing effort, improved execution efficiency, and demonstrated how real-time e-commerce applications can be tested effectively using automation testing frameworks.